

0–k BFS – Текст доклада

Сопровождение к presentation_ru.pdf

Артёмов Макар, Марк Шкут
Сколтех, Алгоритмы 2026

21 мая 2026

Ниже – буквальный текст того, что нужно говорить на каждом слайде. Один раздел – один слайд; зачитываем абзац и переключаем слайд, дойдя до пунктирной линии.

1. Титульный слайд

“Добрый день. Меня зовут Макар, со мной Марк. Мы реализовали на C++17 семейство алгоритмов 0–k BFS для задачи кратчайших путей и сравнили их с Дейкстрой. Заголовочный результат: на графе с 900 миллионами вершин наш алгоритм обрабатывает за две минуты, а Дейкстра за десять. Я расскажу алгоритм, покажу его в работе и пройдуся по замерам. Минут пятнадцать плюс вопросы.”

2. Задача

“Задача – кратчайшие пути из одной вершины во взвешенном графе. Особенность в том, что веса – неотрицательные целые числа, ограниченные сверху небольшим известным k . Это ограничение не искусственное: оно постоянно встречается в планировании пути на сетках, в слоистых графах, при сведениях к 2-SAT, в транспортных сетях с несколькими уровнями стоимости. Конкретная цель – довести размер входа до миллиарда вершин на одной рабочей станции, насколько позволяет железо.”

3. Базовый алгоритм: Дейкстра

“Классический ответ – Дейкстра. Реализация учебная: `std::priority_queue` из пар (расстояние, вершина) и ленивое decrease-key – при каждом улучшении расстояния кладем свежую запись, а при pop пропускаем устаревшие. Сложность $O((V + E) \log V)$. Работает для любых неотрицательных вещественных весов. Множитель $\log V$ платится при каждой успешной релаксации – это и есть цена кучи. Вопрос: можем ли мы избавиться от него, если веса – маленькие целые числа?”

4. Идея: 0–1 BFS

“Простейший случай – веса 0 или 1. Заменяем кучу на deque. Ребро веса 0 кладем в начало, ребро веса 1 – в конец. Deque всё время упорядочен по расстоянию и содержит максимум два различных значения – текущий слой спереди, следующий сзади. Линейное время, линейная память. Тонкость по сравнению с обычным BFS: вершина может быть открыта несколько раз до того, как её расстояние станет окончательным, поэтому проверяем не “посещена/нет”, а $dist[v] + w < dist[u]$.”

5. Обобщение: 0–k BFS

“Теперь обобщаем на веса до k . Наблюдение: релаксация двигает вершину максимум на k шагов вперёд в порядке расстояний. Значит, в любой момент живых слоёв расстояний не больше $k + 1$. Заводим $k + 1$ FIFO-очередь и индексируем их по расстоянию по модулю $k + 1$ – это кольцевой буфер бакетов. Указатель `cur` обходит их. Релаксация кладёт цель в бакет нового расстояния. Когда активный бакет опустеет – `cur` сдвигается вперёд. Устаревшие записи от

прошлых релаксаций остаются в своих старых бакетах и пропускаются при поп. Сложность $O(kV + E)$, память $O(k + E)$ на очереди плюс $O(V)$ на массив расстояний.”

6. Как это выглядит

“Вот оба алгоритма на одной и той же сетке – 200 на 200, $k = 8$. Они выдают одинаковое поле расстояний – волна расходится радиально от центра, форму задаёт случайный рельеф. В заголовках – реально замеренное время. Левая панель, 0-k BFS, закончила за четыре миллисекунды. Правая панель, Дейкстра – за тринадцать с третью миллисекунд – в три с лишним раза дольше. Та же корректность, существенно меньше времени. Сама анимация есть в репозитории, здесь – один кадр.”

7. Под капотом

“Это педагогический центр. На крошечной сетке – десять на десять, $k = 3$ – мы инструментировали алгоритм и выгружаем каждое событие. Справа – четыре реальные очереди $Q[0] - Q[3]$ с их текущим содержимым; числа в клетках – идентификаторы вершин. Слева сетка; цвет клетки показывает, в каком бакете сейчас лежит эта вершина. Красная стрелка `cur` указывает на активный бакет. Видно, как релаксация кладёт вершину в бакет, выбранный новым расстоянием по модулю четыре, и как активный бакет вычерпывается в порядке FIFO до того, как `cur` сдвинется. Это весь секрет алгоритма.”

8. Дизайн под $V \rightarrow 10^9$

“Теперь про инженерию. Чтобы дойти до миллиарда вершин, нельзя хранить рёбра явно: CSR с четырьмя рёбрами на вершину и 32-битными идентификаторами стоил бы уже двадцать гигабайт. Поэтому мы сделали два бэкенда. Универсальный **GraphCSR** для обычных графов и неявный **GridGraph**, который вычисляет четырёх соседей по запросу, без хранения рёбер. Второе – сделали тип расстояния `uint32_t` вместо `uint64_t`, что уменьшает доминирующий массив вдвое. Компромисс допустим, пока $k \cdot V$ помещается в 32 бита, а для наших параметров это так; есть и 64-битный fallback.”

9. Результаты

“Вот цифры – всё на сетке, в один поток. На миллионе вершин уже получаем ускорение в 4.6 раза. На десяти миллионах – 5.6. На ста миллионах при $k = 1$ deque-вариант 0-1 BFS даёт одиннадцатикратное ускорение – это пик. Следующая строка интересна: на половине миллиарда вершин оба алгоритма становятся memory-bandwidth bound и разрыв схлопывается до единицы. Внизу заголовочный результат: на 900 миллионах вершин 0-1 BFS делает работу за 121 секунду, Дейкстра – за 620. Все контрольные суммы расстояний совпадают между алгоритмами – 19 различных инстансов в validation sweep, ноль расхождений.”

10. Масштабирование

“Те же данные, log-log график. Обе оси логарифмические. Дейкстра – верхняя линия, варианты BFS идут параллельно ниже. Постоянный вертикальный зазор в логарифмической шкале означает постоянное мультипликативное ускорение. Справа, при 10^8 и выше, линии начинают сходиться – мы входим в memory-bound режим. Самая нижняя линия всегда – deque-вариант 0-1 BFS, он чисто впереди.”

11. Ограничения

“Честно про границы. Первое: это выигрыш по константе, не асимптотический. Как только k сравним с $\log V$, Дейкстра не хуже. Второе: при V больше полумиллиарда всё упирается в память, и накладные расходы очередей начинают мешать. Третье: содержать $k + 1$ отдельный

FIFO-аллокатор дороже для кэша, чем один deque – именно поэтому при $V \approx 10^9$ наш 0–1 BFS всё ещё выигрывает, а общий 0– k BFS на этом масштабе уже нет. Четвёртое: это не замена Дейкстры в общем случае; для вещественных весов или неизвестного диапазона – использовать Дейкстру.”

12. Дальнейшие шаги

“Четыре пункта в плане. Первый: настоящая индексная d -арная куча с честным decrease-key для baseline – чтобы сравнение было справедливее. Второй: кусочный кольцевой FIFO для 0– k BFS, чтобы закрыть разрыв с 0–1 BFS по кэшу. Третий: выйти из зоны синтетических графов и загрузить реальную дорожную сеть – DIMACS USA с весами, квантованными до $k = 8$. Четвёртый, стретч-цель: вариант Δ -stepping для параллельного контекста.”

13. Спасибо

“Всё это – код, LaTeX-отчёт, анимации, воспроизводимые скрипты сборки – лежит в GitHub-репозитории по ссылке. Готовы к вопросам.”

Вероятные вопросы:

- *Почему 0–1 BFS быстрее 0– k BFS при $k = 1$?* – один `std::deque` против двух `std::queue`: накладные расходы аллокатора и кэша.
- *Как проверяли корректность на запуске с миллиардом вершин?* – суммы расстояний совпадают с Дейкстрой побайтово; сумма фиксирует массив, поэлементное сравнение не требуется.
- *А параллелизм?* – весь Этап 1 в один поток; Δ -stepping – в плане.